Python Programming

From your first line of code to functions and control flow



What you will learn

Learning outcomes • First-year university students

- Explain what Python is and why it is widely used.
- Describe Python’s features, benefits, applications, and limitations.
- Write programs using variables, data types, input(), and print().
- Use operators and type conversion to process values.
- Control program flow with if/elif/else and loops.
- Create reusable functions, including functions with default arguments.

Our route through Unit IV

- 01 Python overview: history, purpose, features, and applications
- 02 Why Python? Benefits, limitations, and comparison with C
- 03 Core syntax: comments, indentation, variables, and data types
- 04 Input/output and type conversion
- 05 Operators and expressions
- 06 Decisions: if, if–else, and if–elif–else
- 07 Repetition: for, while, range(), break, and continue
- 08 Functions: parameters, return values, and default arguments

First, what is a programming language?

- A programming language lets us express instructions that a computer can execute.
- A program is a sequence of instructions designed to solve a problem.
- Examples: Python, C, C++, Java, and JavaScript.
- Different languages make different trade-offs in readability, speed, control, and ecosystem.
- Example problem: calculate the total price of items in a shopping cart.

What is Python?

- Python is a high-level, general-purpose programming language.
- It is designed to make code readable and development productive.
- It supports procedural, object-oriented, and functional programming styles.
- Common uses include automation, web development, data analysis, AI, and scientific work.
- Python is free to use and has a large standard library and community.

A short history of Python

- Created by Guido van Rossum; development began in the late 1980s.
- Python was first publicly released in 1991.
- Python 2.0 was released in 2000.
- Python 3.0 was released in 2008 and introduced important language changes.
- Python 3 is the standard choice for new projects.

Why should we use Python?

- Readable syntax: code often looks close to plain English.
- Less boilerplate: solve the problem without writing as much setup code.
- Fast development: test ideas and build prototypes quickly.
- Broad ecosystem: many libraries already solve common tasks.
- Versatile: one language can support scripts, websites, analysis, and AI.
- Beginner-friendly: focus on logic before complex syntax.

Key features and benefits

Practical reasons Python is useful—not a claim that it is best for every task.

Readable

Clear syntax and indentation

High-level

Hides many low-level details

Interpreted workflow

Usually run without a separate compile step

Cross-platform

Windows, macOS, and Linux

Open source

Free to use; community-developed

Libraries

Standard and third-party packages

Memory management

Automatic memory management

Multi-paradigm

Procedural, OOP, functional

Where is Python used?

Python’s ecosystem makes it useful across many fields.

AI & Machine Learning

Build and evaluate models

PyTorch • TensorFlow • scikit-learn

Data Science

Clean, analyze, and visualize data

Pandas • NumPy • Matplotlib

Web Development

Build server-side applications

Django • Flask • FastAPI

Automation

Automate repetitive tasks

Files • reports • testing

Scientific Computing

Numerical and research workflows

NumPy • SciPy

Education

Teach programming and logic

Small scripts • notebooks

Python has trade-offs too

- Speed: pure Python can be slower than compiled C/C++ for CPU-heavy tasks.
- Memory: some workloads use more memory than lower-level alternatives.
- Deployment: managing package versions and environments takes care.
- Low-level systems: C/C++ may be more suitable for drivers and tight hardware control.
- Choose a language based on the problem, constraints, and ecosystem.

Python vs C: same idea, different amount of syntax

Both can print a greeting; Python typically needs less setup.

C

```c
#include <stdio.h>

int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

Python

```python
print("Hello, World!")
```

Python usually needs less boilerplate. C gives more direct control over low-level details.

Your first Python program

print() displays information on the screen.

```
print("Hello, World!")
```

OUTPUT

```
Hello, World!
```

Comments and indentation

Comments explain code. Indentation defines a block in Python.

```
# This is a comment
marks = 82

if marks >= 40:
    print("Pass")
```

OUTPUT

```
Pass
```

Variables: names that refer to values

- A variable lets us store and reuse a value.
- Example: student_name = "Asha"; marks = 82
- Python determines a variable’s type at runtime based on the value.
- Names may contain letters, digits, and underscores; they cannot start with a digit.
- Names are case-sensitive: marks, Marks, and MARKS are different.
- Prefer descriptive names such as total_price.

Common built-in data types

The type determines what kind of value is represented.

int	Whole numbers	`age = 18`	18
float	Decimal numbers	`price = 99.5`	99.5
str	Text	`name = "Asha"`	Asha
bool	True / False	`is_ready = True`	True

Input and output

input() reads text from the user; print() displays output.

```
name = input("Enter your name: ")
print("Hello,", name)
```

OUTPUT

```
Enter your name: Asha
Hello, Asha
```

Type conversion: input() returns a string

Convert text to numbers before arithmetic.

```
a = input("First number: ")
b = input("Second number: ")
print(a + b)
print(int(a) + int(b))
```

OUTPUT

```
First number: 10
Second number: 20
1020
30
```

Useful print() options

sep changes the separator; end changes what is printed at the end.

```
print("A", "B", "C", sep="-")
print("Loading", end="...")
print("done")
```

OUTPUT

```
A-B-C
Loading...done
```

Arithmetic operators

- + addition: 7 + 3 → 10
- - subtraction: 7 - 3 → 4
- * multiplication: 7 * 3 → 21
- / true division: 7 / 2 → 3.5
- // floor division: 7 // 2 → 3
- % remainder: 7 % 2 → 1
- ** exponentiation: 2 ** 3 → 8

Comparison and logical operators

Comparisons produce Boolean results. and/or/not combine or invert conditions.

```
age = 19
has_id = True
print(age >= 18)
print(age >= 18 and has_id)
print(not has_id)
```

OUTPUT

```
True
True
False
```

Other operators to recognise

- Comparison: ==, !=, >, <, >=, <=
- Assignment: =, +=, -=, *=, /=
- Membership: in, not in — for example, "a" in "cat" is True.
- Identity: is, is not — test whether two references point to the same object.
- Bitwise: &, |, ^, ~, <<, >> — operate on integer bits.
- Do not confuse = (assignment) with == (equality comparison).
- Use == for value equality; use is commonly for identity checks such as value is None.

Decision making: if and if–else

The condition is checked; only the matching block runs.

```
marks = 67

if marks >= 40:
    print("Pass")
else:
    print("Try again")
```

OUTPUT

```
Pass
```

Multiple conditions: if–elif–else

Order conditions from highest threshold to lowest.

```
marks = 82

if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
elif marks >= 40:
    grade = "C"
else:
    grade = "F"
print(grade)
```

OUTPUT

```
B
```

while loop: repeat while condition is true

Update the loop variable so the loop eventually ends.

```
count = 1
while count <= 3:
    print(count)
    count += 1
```

OUTPUT

```
1
2
3
```

for loop and range()

range() generates integers; the stop value is excluded.

```
for i in range(1, 6):
    print(i)

# range(stop) starts at 0
# range(start, stop) excludes stop
```

OUTPUT

```
1
2
3
4
5
```

break, continue, and pass

continue skips to the next iteration; break exits the loop; pass does nothing.

```
for n in range(1, 6):
    if n == 2:
        continue
    if n == 5:
        break
    print(n)

if True:
    pass
```

OUTPUT

```
1
3
4
```

Nested loops: a simple pattern

A loop inside another loop is called a nested loop.

```
for row in range(1, 5):
    print("*" * row)
```

OUTPUT

```
*
**
***
****
```

Functions: reusable blocks of code

def defines a function. return sends a result back to the caller.

```
def square(number):
    return number * number

answer = square(5)
print(answer)
```

OUTPUT

```
25
```

Default arguments

A default value is used when that argument is omitted.

```
def greet(name, greeting="Hello"):
    print(greeting, name)

greet("Asha")
greet("Ravi", "Welcome")
```

OUTPUT

```
Hello Asha
Welcome Ravi
```

print() vs return

- print(value) displays a value to the user.
- return value sends a result back to the code that called the function.
- Returned values can be stored, combined, or passed to another function.
- Example: total = add(4, 5) stores the result returned by add().
- A function without an explicit return statement returns None.

Predict the output

- 1. print(10 // 3, 10 % 3)
- 2. x = "5"; print(x * 2)
- 3. for i in range(2, 6, 2): print(i)
- 4. Loop from 1 to 4; skip i == 3; print the rest.
- 5. def power(x, p=2): return x ** p; print(power(4))
- Try each question before running the program.

Check your answers

- 1. 3 1
- 2. 55 — multiplying a string repeats it.
- 3. 2 then 4
- 4. 1, 2, 4 on separate lines
- 5. 16 — p defaults to 2.
- Discuss why each output appears; do not memorise answers only.

Programming tasks for students

- Write a program to check whether a number is even or odd.
- Read three numbers and print the largest.
- Create a multiplication table for a number entered by the user.
- Calculate the sum of integers from 1 to n.
- Print a right-angled triangle using stars.
- Write a function that returns the cube of a number.
- Write a function power(base, exponent=2) with a default exponent.

Common beginner mistakes

- Forgetting the colon (:) after if, for, while, or def.
- Using inconsistent indentation inside a block.
- Assuming input() automatically returns a number.
- Using = instead of == in a condition.
- Creating an infinite while loop by not updating the condition variable.
- Expecting range(start, stop) to include stop—it does not.
- Confusing print() with return.

Unit IV: quick recap

- Python is a readable, general-purpose language used in many domains.
- Variables refer to values; common types include int, float, str, and bool.
- input() reads text; convert it when numeric input is needed.
- Operators calculate values and form conditions.
- Conditionals choose a path; loops repeat work.
- Functions organise reusable logic; default arguments provide fallback values.
- Best habit: predict the output, run the code, then explain the result.

EXIT TICKET

Can you explain these without notes?

- Why might a team choose Python over C for a quick prototype?
- What is the difference between / and //?
- Why does input("Number: ") return a string?
- What is the difference between break and continue?
- What does return do that print() does not?
- Write a function with one default argument.